

Software Engineering Lab 2: Agile Backlog Creation & Sprint Simulation in Jira

Name: Upadrasta Naga Sarvani

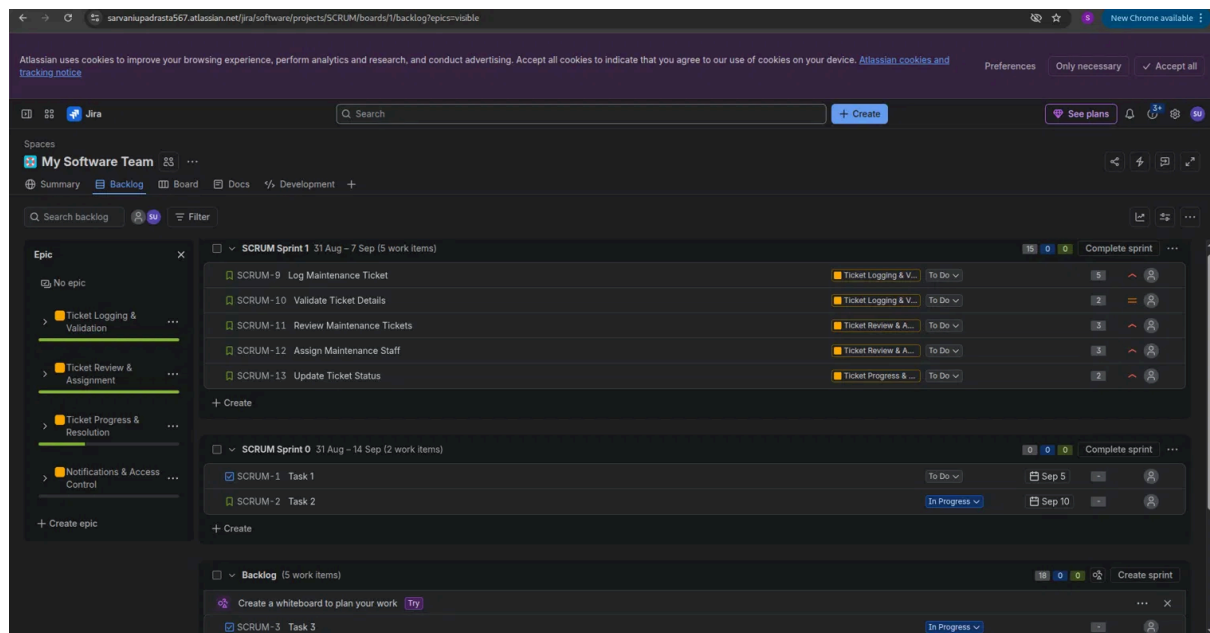
SRN: PES2UG24CS567

Sem-5, Section J

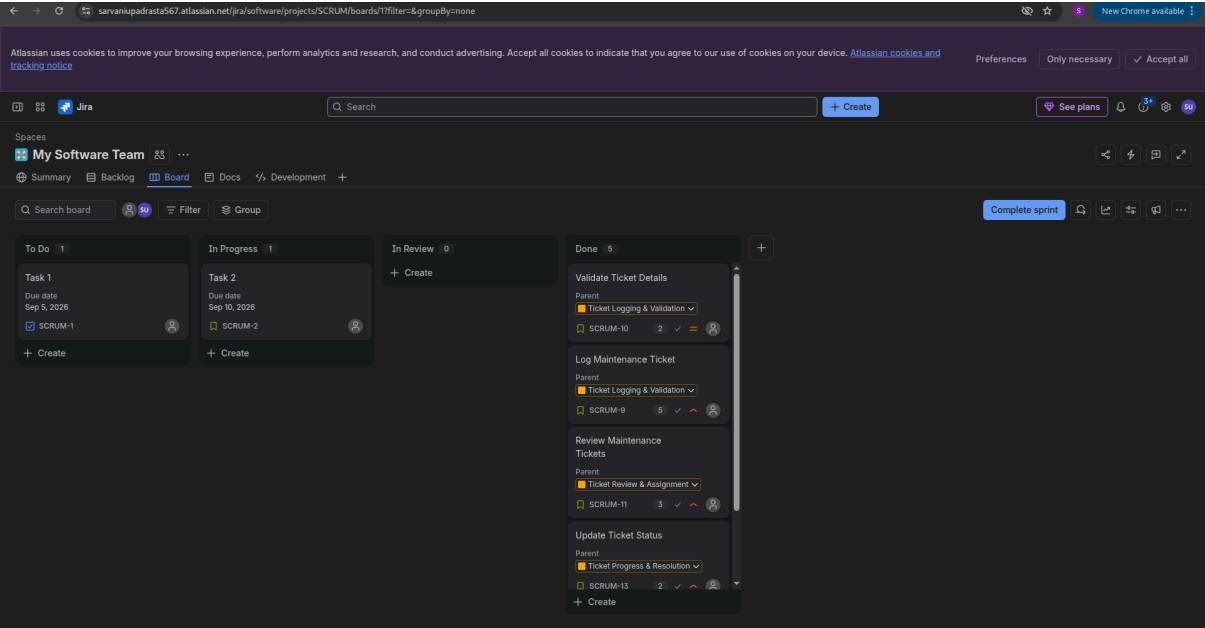
Project: Hostel Maintenance & Issue Ticketing System

Screenshots:

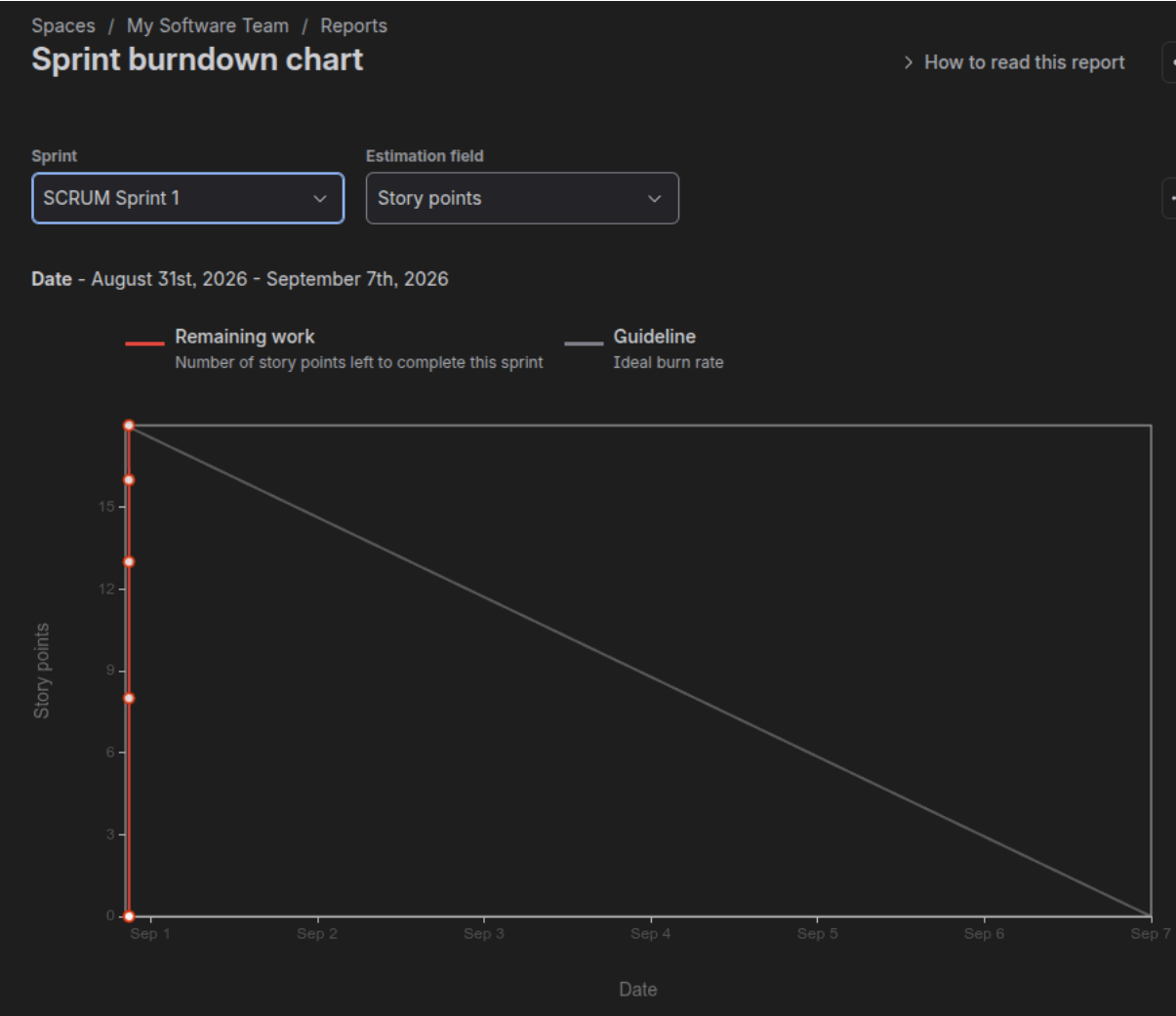
1. Jira backlog with Epics and User Stories:



2. Sprint board (Active Sprint view)



3. Burndown Chart – Sprint 1



4. Burndown Chart – Sprint 2



Reflection Questions

1. Did your estimations reflect the actual effort?

Since this was a simulated sprint rather than actual development work, the story points assigned using the Fibonacci scale reflected my *perceived complexity* of each user story rather than the actual effort required. For example, "Role-Based Access Restriction" was estimated at 8 points because of the complexity of handling multiple user roles, while smaller stories like "Update Ticket Status" were estimated at 2 points. In a real sprint with actual coding and implementation, these estimates would need to be compared with the time and effort that was actually required to complete each

feature. Based on this, the estimates could be adjusted in future sprints according to the team's experience and velocity.

2. Was your backlog well-prioritized?

Yes, the backlog was prioritized based on user impact and the dependency between different features. Core resident-facing functionalities like "Log Maintenance Ticket" and "Validate Ticket Details," along with warden functionalities like "Review Maintenance Tickets" and "Assign Maintenance Staff," were given High priority and placed in Sprint 1 since they represent the essential workflow of the system. Supporting features like resolution notes, closure sign-off, notifications, and access control were placed in Sprint 2. This ordering makes sense because a resident must first be able to log a ticket, and the warden must be able to review and assign it, before further actions like status updates or ticket closure are possible.

3. How did your simulated sprint align with your plan?

The simulated sprints aligned well with the original plan. All the planned stories in both Sprint 1 and Sprint 2 were successfully moved through To Do → In Progress → Done and marked as complete. Since this was a simulation rather than actual development work, there were no unexpected blockers or changes in scope. As a result, all the committed story points were completed within their respective sprints, which meant that the original sprint plan was successfully followed.

4. What insights did the burndown chart give about your team's capacity?

The burndown charts for both sprints showed the "Remaining Work" line dropping sharply to zero almost immediately, rather than decreasing steadily throughout the sprint as the ideal guideline

suggested. This happened because the sprint was simulated in a short session rather than being carried out over actual working days. In a real sprint, a chart like this, where the remaining work drops sharply at the beginning instead of showing gradual and steady progress, could actually be a warning sign. It could indicate that the work was completed too quickly, that the estimates were not accurate, or that the team was not pacing the work sustainably throughout the sprint. This shows why burndown charts are useful not just for tracking whether the work was completed, but also for understanding *how* the work was completed and whether the team's capacity was being used realistically.